

Universidad Tecnológica de Bolívar

ARQUITECTURA DE SOFTWARE

Taller del 2do Corte: Parte 1 (Investigativa)

Fundamentos de Docker y su Importancia dentro de la Arquitectura de Software

Estudiante:
Ramiro José Olivera Herrera - T00084038

Revisado Por:
Mgtr. Luis Giovanny Carreno Ortiz

12 de octubre de 2026

Introducción

Hoy es raro encontrar un proyecto de software serio que no use contenedores. Docker se volvió el estándar para empaquetar aplicaciones y llevarlas de una máquina a otra sin que algo se rompa por el camino. En este informe investigo qué es Docker, cuáles son sus piezas, qué problema resuelve y por qué importa en la Arquitectura de Software, donde cada servicio (una API, una base de datos, un broker de eventos) puede correr como un contenedor.

1. Desarrollo

1.1. ¿Qué es Docker?

Definición. Docker es una plataforma que permite empaquetar una aplicación junto con todo lo que necesita para funcionar (código, librerías, configuración, runtime) en una unidad llamada *contenedor*, de modo que se ejecute igual en cualquier máquina que tenga Docker instalado.

Cómo funciona a alto nivel. Un contenedor es un proceso (o grupo de procesos) aislado del resto del sistema. Ese aislamiento lo da el próprio kernel de Linux, con *namespaces* (cada contenedor ve su propio árbol de procesos, red y sistema de archivos) y *cgroups* (limitan cuánta CPU y memoria puede usar). Por eso el contenedor **comparte el kernel del sistema operativo del host** en lugar de traer uno propio: solo lleva lo que está por encima del kernel.

Contenedor vs. máquina virtual. Una máquina virtual (VM) emula hardware completo y arranca su propio sistema operativo, con su propio kernel, encima de un hipervisor. Un contenedor no emula nada: reutiliza el kernel del host. Por eso es más ligero.

	Máquina virtual	**Contenedor**
Sistema operativo	Uno completo por VM	Comparte el del host
Tamaño	Gigabytes	Megabytes
Arranque	Minutos	Segundos o menos
Aislamiento	Muy fuerte (hardware)	Fuerte, a nivel de procesos
Consumo	Alto	Bajo

Tabla 1: Comparación entre máquina virtual y contenedor.

1.2. ¿Cuáles son sus componentes?

- **CLI (`docker`):** la herramienta de línea de comandos con la que le damos órdenes a Docker (`docker run`, `docker build`, etc.). No ejecuta nada por sí sola, solo envía peticiones.
- **Daemon (`dockerd`):** el motor que corre en segundo plano. Recibe las peticiones del CLI y hace el trabajo: construye imágenes, crea y ejecuta contenedores, administra redes y volúmenes.

- **Imagen:** una plantilla de solo lectura, hecha por capas, que contiene todo lo necesario para ejecutar una aplicación. Es la "receta ya cocinada".
- **Contenedor:** una instancia en ejecución de una imagen. De una misma imagen se pueden crear muchos contenedores.
- **Dockerfile:** un archivo de texto con las instrucciones para construir una imagen (`FROM`, `COPY`, `RUN`, `CMD`, etc.).
- **Registro (Docker Hub):** un almacén de imágenes donde se descargan (`pull`) y se publican (`push`). Docker Hub es el público por defecto, pero también existen registros privados.

La relación entre ellos se resume en el siguiente esquema:

```
Dockerfile --(docker build)--> Imagen --(docker run)--> Contenedor
                           ^ |
                docker pull | | docker push
                            | v
                      Registro (Docker Hub)

CLI (docker) --> Daemon (dockerd) --> ejecuta todo lo anterior
```

Se escribe un **Dockerfile**, con `docker build` se genera una **imagen**, y con `docker run` se crea un **contenedor** a partir de ella. El **registro** entra en dos momentos: para *bajar* imágenes que otros ya construyeron (como `mysql:8`) y para *subir* las nuestras y compartirlas con el equipo o con el servidor de producción.

1.3. Principales funcionalidades

1. **Portabilidad.** La misma imagen corre igual en mi laptop, en la de un compañero y en un servidor en la nube. *Ejemplo:* una API en Node.js empaquetada en una imagen funciona igual en Windows, Linux o macOS con Docker.
2. **Aislamiento.** Cada aplicación vive en su propio entorno y no interfiere con las demás. *Ejemplo:* puedo correr un proyecto con Node 14 y otro con Node 18 al mismo tiempo, sin conflictos de versiones.
3. **Versionado de imágenes.** Las imágenes se identifican con etiquetas (*tags*), lo que permite fijar y cambiar versiones de forma controlada. *Ejemplo:* `mysql:8` o `node:18`; si una actualización falla, vuelvo a la etiqueta anterior.
4. **Reproducibilidad del entorno.** Todo el equipo usa exactamente el mismo entorno, definido en un Dockerfile o un `docker-compose.yml`. *Ejemplo:* un compañero nuevo clona el repositorio, ejecuta `docker compose up` y tiene la API y la base de datos funcionando sin instalar nada a mano.
5. **Escalabilidad.** Se pueden arrancar varias instancias de un mismo servicio para repartir carga. *Ejemplo:* levantar tres contenedores de la misma API detrás de un balanceador cuando aumenta el tráfico.

6. **Base para CI/CD.** La imagen construida en el pipeline es la misma que se prueba y se despliega. *Ejemplo:* GitHub Actions construye la imagen en cada *push*, corre las pruebas dentro de ella y la publica en un registro.

1.4. Principales comandos

a) Tabla de comandos

Comando	¿Qué hace?	Ejemplo
`docker pull`	Descarga una imagen desde un registro	`docker pull mysql:8`
`docker run`	Crea y arranca un contenedor desde una imagen	`docker run -d --name web -p 8080:80 nginx`
`docker ps`	Lista de todos los contenedores en ejecución	`docker ps`
`docker ps -a`	Lista todos los contenedores, también los detenidos	`docker ps -a`
`docker logs`	Muestra la salida (logs) de un contenedor	`docker logs -f web`
`docker exec`	Ejecuta un comando dentro de un contenedor en marcha	`docker exec -it web bash`
`docker stop` /`start`	Detiene/ vuelve a iniciar un contenedor existente	`docker stop web` y `docker start web`
`docker rm`	Esto elimina un contenedor (detenido)	`docker rm web`
`docker images`	Organiza en una lista las imágenes locales	`docker images`
`docker rmi`	Sirve para eliminar una o más imágenes locales	`docker rmi nginx`
`docker build`	Construye una imagen desde un Dockerfile	`docker build -t mi-api:1.0 .`
`docker compose up`	Levanta todos los servicios definidos en `docker-compose.yml`	`docker compose up -d`

Tabla 2: Principales comandos de Docker.

b) Tres comandos en detalle

`docker run`**.** Es el comando más usado porque crea y arranca el contenedor. Sus parámetros comunes son:

- `-d`: ejecuta el contenedor en segundo plano (*detached*).
- `-p 8080:80`: publica el puerto 80 del contenedor en el puerto 8080 de mi máquina.

- `-e VAR=valor`: define variables de entorno.
- `-v datos:/var/lib/mysql`: monta un volumen para que los datos persistan.
- `--name`: asigna un nombre al contenedor.

Situación de uso: levantar una base de datos para desarrollar:

```
docker run -d --name mi-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secreto -v datos
    :/var/lib/mysql mysql:8
```

Sin `-v`, los datos se perderían al eliminar el contenedor.

`docker exec`**.** Permite ejecutar un comando dentro de un contenedor que ya está corriendo. Parámetros comunes: `-it` (modo interactivo con terminal) y el comando a ejecutar al final. *Situación de uso:* entrar a la base de datos para revisar una tabla: `docker exec -it mi-db mysql -u root -p`.

`docker build`**.** Construye una imagen a partir de un Dockerfile. Parámetros comunes: `-t nombre:etiqueta` (nombre y versión de la imagen) y el contexto de construcción (normalmente `.`, la carpeta actual). También existe `-f` para indicar un Dockerfile con otro nombre. *Situación de uso:* cuando termino una nueva versión de mi API y quiero empaquetarla: `docker build -t mi-api:1.1 .`, para luego publicarla o ejecutarla.

1.5. ¿Qué problema resuelve?

Es el clásico **"en mi máquina funciona"**. Una aplicación puede funcionar perfecto donde se desarrolló y fallar en otra máquina porque el entorno es distinto. Las causas típicas son:

- **Versiones de dependencias:** otra versión de Node, Python o de una librería.
- **Sistema operativo:** diferencias entre Windows, Linux y macOS (rutas, permisos, librerías del sistema).
- **Configuración:** variables de entorno, puertos o archivos que faltan.
- **Instalación manual:** pasos que alguien hizo a mano, nadie documentó y no se pueden repetir con fiabilidad.

Antes vs. después (ejemplo). Un equipo de tres personas desarrolla una API con MySQL.

- *Antes:* cada uno instala MySQL a mano; uno tiene la versión 5.7, otro la 8 y otro tiene el puerto ocupado. La API funciona en dos computadores y falla en el tercero, y se pierde medio día averiguando por qué. En el servidor de producción, otra sorpresa.
- *Después:* el repositorio incluye un Dockerfile y un `docker-compose.yml` que fijan `mysql:8` y la versión de Node. Cada integrante ejecuta `docker compose up` y obtiene un entorno idéntico. Lo que se probó es lo que se despliega.

1.6. ¿Por qué es importante en la Arquitectura de Software?

- **Un servicio, un contenedor.** Poner cada componente (API, base de datos, broker de eventos) en su propio contenedor da aislamiento, permite actualizarlos, reiniciarlos o reemplazarlos de forma independiente, y facilita ubicar fallos. Si cae el servicio de notificaciones, no tiene por qué caer el de pagos.

- **Despliegue consistente.** Como la imagen es la misma, el entorno es idéntico en desarrollo, pruebas y producción. Se reducen los errores que solo aparecen al desplegar.
- **Escalabilidad y nube.** Si un servicio recibe mucha carga, se arrancan más contenedores de él. Además, como un contenedor corre en cualquier lugar con Docker, mover el sistema entre proveedores de nube o a orquestadores como Kubernetes es mucho más simple.
- **Relación con los estilos arquitectónicos.**
  - *Microservicios:* cada microservicio es naturalmente un contenedor independiente.
  - *SOA:* los servicios pueden empaquetarse y desplegarse por separado, con contratos bien definidos.
  - *Arquitectura orientada a eventos:* el broker (por ejemplo RabbitMQ o Kafka) y los productores y consumidores corren como contenedores que se comunican por la red.

En resumen, Docker no define la arquitectura, pero es la herramienta concreta que hace viable y repetible la mayoría de estos estilos.

Conclusión

Con esta investigación entendí que Docker no es solo "una herramienta más", sino la pieza que vuelve práctico separar un sistema en servicios y desplegarlo de forma confiable. Lo que más me llamó la atención es que resuelve un problema que he vivido en trabajos grupales: cuando un proyecto corre en el computador de uno y no en el de otro. Lo usaría en mis proyectos de grupo, por ejemplo en una plataforma con base de datos MySQL, backend y frontend: con un `docker-compose.yml` todos los integrantes tendrían el mismo entorno desde el primer día, sin perder tiempo en instalaciones, y el despliegue final sería mucho más simple.

Referencias

[1] Docker Inc. *Docker overview*. Documentación oficial de Docker. `https://docs.docker.com/get-started/docker-overview/`

[2] Docker Inc. *Docker CLI reference*. Documentación oficial de Docker. `https://docs.docker.com/reference/cli/docker/`

[3] Docker Inc. *What is a container?* Docker Resources. `https://www.docker.com/resources/what-container/`

[4] Merkel, D. (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014(239). `https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment`